Algoritmo e Laboratório de Programação II

Aula 5 – Ponteiros - parte 1 (Nível Avançado)

Roteiro

- O que são ponteiros?
- Por que os ponteiros são usados?
- Ponteiros variáveis
- Ponteiros constantes e o operador de endereço &
- Passando argumentos por referência com ponteiros
- Variáveis que armazenam endereços
- O operador indireto *
- Passando endereços para a função
- Permutação do valor de duas variáveis
- Ponteiros sem funções
- Ponteiros e variáveis apontadas
- Operações com ponteiros
- A unidade adotada em operações com ponteiros

Introdução

- Ponteiros são a ferramenta mais poderosa oferecida pela linguagem C, considerados pela maioria dos programadores como um dos tópicos mais difíceis nessa linguagem.

Introdução

- Há dois principais motivos para isso:
  - Primeiro, os conceitos embutidos em ponteiros podem ser novos para muitos programadores, visto que não são comuns em linguagens de alto nível;
  - Segundo, os símbolos usados para notação de ponteiros em C não são tão claros quanto poderiam ser. Por exemplo, o mesmo símbolo é usado para duas finalidades diferentes.

Introdução

- Conceitualmente, os ponteiros podem ser difíceis, mas não muito.
- Nosso objetivo neste capítulo é expor tão claramente quanto possível como trabalhar com ponteiros.

Roteiro

- O que são ponteiros?
- Por que os ponteiros são usados?
- Ponteiros variáveis
- Ponteiros constantes e o operador de endereço &
- Passando argumentos por referência com ponteiros
- Variáveis que armazenam endereços
- O operador indireto *
- Passando endereços para a função
- Permutação do valor de duas variáveis
- Ponteiros sem funções
- Ponteiros e variáveis apontadas
- Operações com ponteiros
- A unidade adotada em operações com ponteiros

O Que São Ponteiros?

- O nome de uma variável indica o que está armazenado nela.
- O endereço de uma variável é um ponteiro.
- Seu valor indica em que parte da memória do computador a variável está alocada.
- Ponteiros proporcionam um modo de acesso à variável sem referenciá-la diretamente (modo indireto de acesso).

Por Que os Ponteiros São Usados?

- Para dominar a linguagem C, é essencial dominar ponteiros, que são utilizados em situações em que o uso do nome de uma variável não é permitido ou é indesejável.
- Algumas razões para o uso de ponteiros são:
  - Fornecem maneiras com as quais as funções podem realmente modificar os argumentos que recebem (passagem por referência);
  - Passar matrizes e strings mais convenientemente de uma função para outra (usá-los no ligar de matrizes);

Por Que os Ponteiros São Usados?

- Manipular os elementos de matrizes mais facilmente, por meio da movimentação de ponteiros para elas (ou parte delas), no lugar de índices entre colchetes;
- Criar estruturas de dados complexas, como listas encadeadas e árvores binárias, em que um item deve conter referências a outro;
- Alocar e desalocar memória dinamicamente do sistema;
- Passar para uma função o endereço de outra função.

Roteiro

- O que são ponteiros?
- Por que os ponteiros são usados?
- Ponteiros variáveis
- Ponteiros constantes e o operador de endereço &
- Passando argumentos por referência com ponteiros
- Variáveis que armazenam endereços
- O operador indireto *
- Passando endereços para a função
- Permutação do valor de duas variáveis
- Ponteiros sem funções
- Ponteiros e variáveis apontadas
- Operações com ponteiros
- A unidade adotada em operações com ponteiros

Ponteiros Variáveis

- Em C, há um tipo especial de variável, concebida para conter o endereço de outra variável, que se chama ponteiro variável.
- Um ponteiro variável armazena um endereço de memória, que é a localização de outra variável.
- Dizemos que uma variável aponta para outra variável quando a primeira contém o endereço da segunda.

Ponteiros Variáveis

- A memória de seu computador é dividida em bytes, numerados de zero até o limite de memória de sua máquina.
- Esses números são chamados endereços de bytes.
- Um endereço é uma referência que o computador usa para localizar variáveis.
- Toda variável ocupa uma certa localização na memória, e seu endereço é o primeiro byte ocupado por ela.

Ponteiros Variáveis

- Nossos programas, quando carregados para a memória, ocupam uma certa parte dela.
- Desta forma, toda variável e toda função de nosso programa começa em um endereço particular, que é chamado de endereço de variável ou da função.

Roteiro

- O que são ponteiros?
- Por que os ponteiros são usados?
- Ponteiros variáveis
- Ponteiros constantes e o operador de endereço &
- Passando argumentos por referência com ponteiros
- Variáveis que armazenam endereços
- O operador indireto *
- Passando endereços para a função
- Permutação do valor de duas variáveis
- Ponteiros sem funções
- Ponteiros e variáveis apontadas
- Operações com ponteiros
- A unidade adotada em operações com ponteiros

Ponteiros Constantes e o Operador de Endereços &

- Para conhecer o endereço ocupado por uma variável usamos o operador de endereços &.
- O resultado da operação é um ponteiro constante.
- Eis um pequeno programa que mostra o seu uso:

Ponteiros Constantes e o Operador de Endereços &

Prog1.c

```c
/* Prog1.c */
/* Mostra ponteiros constantes */
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i,j,k;

    printf("Endereco de i = %p\n", &i); /* %p para ponteiros */
    printf("Endereco de j = %p\n", &j);
    printf("Endereco de k = %p\n", &k);

    system("PAUSE");
    return 0;
}
```

Ponteiros Constantes e o Operador de Endereços &

- Esse programa declara três variáveis e imprime o endereço de cada uma delas.
- O endereço ocupado por essas variáveis dependerá de vários fatores, como o tamanho do sistema operacional, se há ou não outros programas residentes na memória, etc.
- Por essas razões você pode encontrar números diferentes quando executar o programa.

Ponteiros Constantes e o Operador de Endereços &

- Decidimos imprimir os endereços em hexadecimal.
- Esses endereços são constantes.

Roteiro

- O que são ponteiros?
- Por que os ponteiros são usados?
- Ponteiros variáveis
- Ponteiros constantes e o operador de endereço &
- Passando argumentos por referência com ponteiros
- Variáveis que armazenam endereços
- O operador indireto *
- Passando endereços para a função
- Permutação do valor de duas variáveis
- Ponteiros sem funções
- Ponteiros e variáveis apontadas
- Operações com ponteiros
- A unidade adotada em operações com ponteiros

Passando Argumentos por Referência com Ponteiros

- Uma função pode receber diversos argumentos, mas só consegue retornar um único valor por meio do comando return.
- Como fazer para que uma função retorne mais de um valor para a função chamadora?
- Usando ponteiros.

Passando Argumentos por Referência com Ponteiros

- Há duas maneiras de passar argumentos para uma função: por valor e por referência por meio de ponteiros.

Passando Argumentos por Referência com Ponteiros

- Para que uma função retorne mais de um valor para a função chamadora, devemos seguir dois passos:
  - Primeiro, a função chamadora passa os endereços das suas variáveis usando o operador de endereços, os quais indicam variáveis nas quais queremos que a função chamada coloque os valores que devem ser retornados. Quando passamos endereços de variáveis, a função pode alterar a variável original.

Passando Argumentos por Referência com Ponteiros

  - Segundo, a função chamada deverá criar variáveis para armazenar os endereços que estiver recebendo, enviados pela função chamadora. Essas variáveis são ponteiros variáveis.

- Vamos mostrar um exemplo:

Passando Argumentos por Referência com Ponteiros

Prog2.c

```c
/* Prog2.c */
/* Mostra o uso de ponteiros variáveis */
#include <stdio.h>
#include <stdlib.h>

void reajusta20( float *, float *); /* protótipo */

int main()
{
    float val_preco, val_reaj;
    do
    {
        printf("\nInsira o preco atual: ");
        scanf("%f", &val_preco);
        reajusta20(&val_preco, &val_reaj); /* Enviando enderecos */
        printf("\nO preco novo é %.2f\n", val_preco);
        printf("O aumento foi de %.2f\n", val_reaj);
    }
    while( val_preco != 0.0);

    system("PAUSE");
    return 0;
}
```

Passando Argumentos por Referência com Ponteiros

```
25  /* reajusta20() */
26  /* Reajusta o preco em 20% */
27  void reajusta20(float *preco, float *reajuste) /* Recebendo ponteiros */
28  {
29      *reajuste = *preco * 0.2;
30      *preco *= 1.2;
31  }
```

Passando Argumentos por Referência com Ponteiros

- O programa solicita ao usuário que insira o preço atual de uma certa mercadoria.
- Em seguida, modifica o preço para um valor aumentado em 20% e calcula o valor do aumento.

Roteiro

- O que são ponteiros?
- Por que os ponteiros são usados?
- Ponteiros variáveis
- Ponteiros constantes e o operador de endereço &
- Passando argumentos por referência com ponteiros
- Variáveis que armazenam endereços
- O operador indireto *
- Passando endereços para a função
- Permutação do valor de duas variáveis
- Ponteiros sem funções
- Ponteiros e variáveis apontadas
- Operações com ponteiros
- A unidade adotada em operações com ponteiros

Variáveis que armazenam endereços

- Você já usou variáveis para armazenar números inteiros, caracteres, números em ponto flutuante, etc.
- Endereços são armazenados, de modo semelhante, em variáveis com um tipo específico para esse fim.
- Uma variável que armazena um endereço é chamada ponteiro.

Variáveis que armazenam endereços

- Uma variável que armazena o endereço de uma variável do tipo float é do tipo float*.
- O asterisco faz parte do nome do tipo e indica ponteiro para.
- A função reajusta20() recebe como argumento dois ponteiros para float.

Variáveis que armazenam endereços

- O seu protótipo

```
void reajusta20( float *, float *); /* protótipo */
```

- Indica que serão criadas variáveis do tipo float* para receber os endereços das variáveis passadas como argumentos pela função main().

Variáveis que armazenam endereços

```
25  /* reajusta20() */
26  /* Reajusta o preco em 20% */
27  void reajusta20(float *preco, float *reajuste)
28  {
29      *reajuste = *preco * 0.2;
30      *preco *= 1.2;
31  }
```

- A declaração dos parâmetros da função reajusta20() indica que *preco e *reajuste são do tipo float e que preco e reajuste são do tipo float* (ponteiros para variáveis float).

Variáveis que armazenam endereços

- Na verdade, usar *preco é uma maneira indireta de usar a variável val_preco de main().
- Toda alteração feita em *preco afetará diretamente val_preco.
- Como preco contém o endereço da variável val_preco, dizemos que preco aponta para val_preco.

Roteiro

- O que são ponteiros?
- Por que os ponteiros são usados?
- Ponteiros variáveis
- Ponteiros constantes e o operador de endereço &
- Passando argumentos por referência com ponteiros
- Variáveis que armazenam endereços
- O operador indireto *
- Passando endereços para a função
- Permutação do valor de duas variáveis
- Ponteiros sem funções
- Ponteiros e variáveis apontadas
- Operações com ponteiros
- A unidade adotada em operações com ponteiros

O operador Indireto *

- O operador indireto * é unário e opera sobre um endereço ou ponteiro.
- O resultado da operação é o nome da variável localizada nesse endereço (apontada).
- O nome da variável representa o seu valor ou conteúdo.
- Em outras palavras, resulta o valor da variável apontada.

O operador Indireto *

- **O operador de endereços & opera sobre o nome de uma variável e resulta o seu endereço, já o operador indireto * opera sobre o endereço de uma variável e resulta seu nome.**

Roteiro

- O que são ponteiros?
- Por que os ponteiros são usados?
- Ponteiros variáveis
- Ponteiros constantes e o operador de endereço &
- Passando argumentos por referência com ponteiros
- Variáveis que armazenam endereços
- O operador indireto *
- Passando endereços para a função
- Permutação do valor de duas variáveis
- Ponteiros sem funções
- Ponteiros e variáveis apontadas
- Operações com ponteiros
- A unidade adotada em operações com ponteiros

Passando Endereços para a Função

- A nossa função main() chama a função reajusta20() por meio da seguinte instrução:

```
reajusta20(&val_preco, &val_reaj);
```

- Os endereços das variáveis val_preco e val_reaj são acessados por meio do operador de endereços e enviados como argumentos para a função reajusta20().
- A função reajusta20() recebe esses valores em ponteiros (tipos de variáveis para armazenar endereços).

Passando Endereços para a Função

- Para que essa função possa acessar as variáveis apontadas, deverá não somente conhecer os seus endereços como também os seus tipos.
- O endereço informa o primeiro byte ocupado pela variável;
- É necessário saber quantos bytes estão ocupados por ela e qual é a forma de armazenamento para se ter acesso ao seu conteúdo.

Passando Endereços para a Função

- O tipo de variável apontada é dado na declaração do ponteiro:

```
float *preco, float *reajuste
```

- Assim, float é do tipo da variável apontada por preco.
- A função pode então usar instruções como:

```
*reajuste = *preco * 0.2;
*preco *= 1.2;
```

- Que modificam indiretamente as variáveis val_preco e val_reaj.

Passando Endereços para a Função

- É indireto porque a função não tem acesso direto aos nomes dessas variáveis, e sim aos seus endereços.
- A conclusão é que main() acessa diretamente as variáveis val_preco e val_reaj, enquanto reajusta20() as acessa via operador indireto.
- Uma vez conhecidos os endereços e os tipos das variáveis como também ler o valor já armazenado nelas.

Passando Endereços para a Função

- Ponteiros podem ser usados para que a função passe valores para o programa chamador, mas também para que o programa chamador passe valores para a função.
- Na instrução:

```
*reajuste = *preco * 0.2;
```

- Estamos usando o valor original da variável val_preco, por meio de *preco, multiplicando esse valor por 0,2 e atribuindo o resultado à variável val_reaj, indiretamente por meio de *reajuste.

Roteiro

- O que são ponteiros?
- Por que os ponteiros são usados?
- Ponteiros variáveis
- Ponteiros constantes e o operador de endereço &
- Passando argumentos por referência com ponteiros
- Variáveis que armazenam endereços
- O operador indireto *
- Passando endereços para a função
- Permutação do valor de duas variáveis
- Ponteiros sem funções
- Ponteiros e variáveis apontadas
- Operações com ponteiros
- A unidade adotada em operações com ponteiros

Permutação do Valor de Duas Variáveis

- Por padrão, a linguagem C passa argumento para funções usando “chamada por valor”; a função chamada não pode alterar diretamente uma variável da função chamadora.
- Por exemplo, uma rotina de ordenação poderia querer permutar dois elementos fora de ordem por meio de uma função chamada troca().

Permutação do Valor de Duas

Prog3.c

```c
/* Prog3.c */
/* Função Troca */
#include <stdio.h>
#include <stdlib.h>

void troca(int, int); /* protótipo */

int main()
{
    int v1, v2;

    printf("\nInsira o Primeiro Valor: ");
    scanf("%d", &v1);

    printf("\nInsira o Segundo Valor: ");
    scanf("%d", &v2);

    troca(v1, v2);

    printf("\nV1: %d; V2: %d", v1, v2);

    return 0;
}
```

Permutação do Valor de Duas Variáveis

```
25  /* Efetua a troca */
26  void troca(int x, int y)
27  {
28      int temp;
29      temp = x;
30      x = y;
31      y = temp;
32  }
```

Permutação do Valor de Duas Variáveis

- Por causa da chamada por valor, troca() não pode afetar os argumentos x e y da função que chama.
- Ponteiros resolvem o problema.
- O programa chamador passa os endereços dos valores a serem permutados
- E na função troca(), os parâmetros são declarados como ponteiros, e as variáveis atuais são acessadas por meio deles.

Permutação do Valor de Duas Variáveis

Prog4.c

```c
/* Prog4.c */
/* Função Troca */
#include <stdio.h>
#include <stdlib.h>

void troca(int * , int *); /* protótipo */

int main()
{
    int v1, v2;

    printf("\nInsira o Primeiro Valor: ");
    scanf("%d", &v1);

    printf("\nInsira o Segundo Valor: ");
    scanf("%d", &v2);

    troca(&v1, &v2);

    printf("\nV1: %d; V2: %d", v1, v2);

    return 0;
}
```

Permutação do Valor de Duas Variáveis

```
25  /* Efetua a troca */
26  void troca(int *x, int *y)
27  {
28      int temp;
29      temp = *x;
30      *x = *y;
31      *y = temp;
32  }
```

Roteiro

- O que são ponteiros?
- Por que os ponteiros são usados?
- Ponteiros variáveis
- Ponteiros constantes e o operador de endereço &
- Passando argumentos por referência com ponteiros
- Variáveis que armazenam endereços
- O operador indireto *
- Passando endereços para a função
- Permutação do valor de duas variáveis
- Ponteiros sem funções
- Ponteiros e variáveis apontadas
- Operações com ponteiros
- A unidade adotada em operações com ponteiros

Ponteiros Sem Funções

- O exemplo que apresentamos usou ponteiros como argumentos de funções.
- O programa seguinte cria ponteiros como variáveis automáticas dentro da função main().

Ponteiros Sem Funções

Prog5.c

```c
/* Prog5.c */
/* Mostra o uso de ponteiros declarados
   dentro da função main() */
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int x=4, y=7;
    int *px,*py;

    printf("&x = %p\t  x = %d\n", &x , x);
    printf("&y = %p\t  y = %d\n", &y , y);

    px = &x;
    py = &y;

    printf("px = %p\t*px = %d\n", px,*px);
    printf("py = %p\t*py = %d\n", py,*py);

    system("PAUSE");
    return 0;
}
```

Ponteiros Sem Funções

- A instrução

```
int *px,*py;
```

- Declara px e py como ponteiros para variáveis int.
- Quando um ponteiro não é inicializado na instrução de sua declaração, o compilador inicializa-o com o endereço zero (NULL).

Ponteiros Sem Funções

- A linguagem C garante que NULL não é um endereço válido, então, antes de usá-los, devemos atribuir a eles algum endereço válido, isto é feito pelas instruções:

```
px = &x;
py = &y;
```

- Um ponteiro pode ser inicializado na mesma instrução de sua declaração:

```
int *px = &x,*py = &y;
```

Ponteiros Sem Funções

- Observe que estamos atribuindo &x e &y a px e py respectivamente, e não a *px e *py.
- O asterisco na declaração faz parte do nome do tipo.

Ponteiros Sem Funções

- C permite ponteiros de qualquer tipo, e as sintaxes de declaração possíveis são as seguintes:
  - Operador indireto junto ao nome da variável.

```
int *p        /* Ponteiro int */
char *p       /* Ponteiro char */
String *p     /* Ponteiro para um tipo */
              /* Definido pelo usuário */
```

  - Operador indireto junto ao nome do tipo.

```
int* p        /* Ponteiro int */
char* p       /* Ponteiro char */
String* p     /* Ponteiro para um tipo */
              /* Definido pelo usuário */
```

Ponteiros Sem Funções

- Se vários ponteiros são declarados numa mesma instrução, o tipo deve ser inserido somente uma vez; o asterisco, todas as vezes.

```
int* p, * p1, * p2;
int *p, *p1, *p2;
```

- Inicializando ponteiros.

```
int i;
int *pi=&i;
int *pj, *pi=&i, *px;
```

- Ponteiros e variáveis simples declarados numa única instrução.

```
int *p, i, *q;
int *px = &x, i, j = 5, *q;
```

Roteiro

- O que são ponteiros?
- Por que os ponteiros são usados?
- Ponteiros variáveis
- Ponteiros constantes e o operador de endereço &
- Passando argumentos por referência com ponteiros
- Variáveis que armazenam endereços
- O operador indireto *
- Passando endereços para a função
- Permutação do valor de duas variáveis
- Ponteiros sem funções
- Ponteiros e variáveis apontadas
- Operações com ponteiros
- A unidade adotada em operações com ponteiros

Ponteiros E Variáveis Apontadas

- Você pode usar ponteiros para executar qualquer operação na variável apontada.
- Nesse programa, usamos o ponteiro para atribuir um valor à variável x; em seguida, usamos novamente o ponteiro para atribuir esse valor a y.

Ponteiros E Variáveis Apontadas

Prog6.c

```c
/* Prog6.c */
/* Mostra a inicialização do ponteiro */
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int x,y;
    int *px = &x;    /* Inicializa px com o endereco de x */

    *px = 14;    /* O mesmo que x = 14 */
    y = *px;     /* O mesmo que y = x  */

    printf("y = %d\n", y);

    system("PAUSE");
    return 0;
}
```

Ponteiros E Variáveis Apontadas

- O operador indireto, precedendo o nome do ponteiro em instruções como:

```
y = *px;      /* O mesmo que y = x   */
```

- Resulta o nome da variável apontada.

Roteiro

- O que são ponteiros?
- Por que os ponteiros são usados?
- Ponteiros variáveis
- Ponteiros constantes e o operador de endereço &
- Passando argumentos por referência com ponteiros
- Variáveis que armazenam endereços
- O operador indireto *
- Passando endereços para a função
- Permutação do valor de duas variáveis
- Ponteiros sem funções
- Ponteiros e variáveis apontadas
- Operações com ponteiros
- A unidade adotada em operações com ponteiros

Operações com Ponteiros

- C permite várias operações básicas com ponteiros.
- Nosso próximo exemplo mostra essas possibilidades.
- O programa imprime os resultados de cada operação, o valor do ponteiro, o valor da variável apontada e o endereço do próprio ponteiro.

Operações com Ponteiros

Prog7.c

```c
/* Prog7.c */
/* Mostra as operações possíveis com ponteiros */
#include <stdio.h>
#include <stdlib.h>

int main()
{
    unsigned int x=5, y=6;
    unsigned int *px, *py;

    px = &x;     /* Atribuições */
    py = &y;

    if( px < py) /* Comparações */
        printf("py-px= %u\n", (py-px));/* Subtracao */
    else
        printf("px-py= %u\n", (px-py));

    printf("px = %p", px);
    printf(", *px = %u", *px);  /* Op.Indireto */
    printf(", &px = %p\n", &px);    /* Op.Enderecos */
```

Operações com Ponteiros

```
23      printf("py = %p", py);
24      printf(", *py = %u", *py);
25      printf(", &py = %p\n", &py);
26
27      py++;   /* Incremento */
28
29      printf("py = %p", py);
30      printf(", *py = %u", *py);
31      printf(", &py = %p\n", &py);
32
33      px = py + 5; /* Somar inteiros */
34
35      printf("px = %p", px);
36      printf(", *px = %u", *px);
37      printf(", &px = %p\n", &px);
38
39      printf("px-py= %u\n", (px-py));
40
41      system("PAUSE");
42      return 0;
43  }
```

Operações com Ponteiros

- Atribuição:
  - Um endereço pode ser atribuído a um ponteiro desde que este seja do mesmo tipo daquele.
  - Geralmente, fazemos isso usando o operados de endereços (&) junto ao nome de uma variável.
  - Em nosso exemplo, atribuímos a px o endereço de x e a py o endereço de y.

```
px = &x;
py = &y;
```

Operações com Ponteiros

- Operação Indireta:
  - O operador indireto *, precedendo o nome do ponteiro, resulta no nome da variável apontada.
- Trazendo o Endereço do Ponteiro:
  - Como todas as variáveis, os ponteiros têm um endereço e um valor.
  - O operador &, precedendo o nome do ponteiro, resulta na posição de memória em que o ponteiro está localizado.
  - O nome do ponteiro indica o valor contido nele, isto é, o endereço para o qual ele aponta (o endereço da variável apontada).

Operações com Ponteiros

- Incrementando um ponteiro:
  - Podemos incrementar um ponteiro por meio de adição regular ou do operador de incremento.
  - Incrementar um ponteiro acarreta sua movimentação para o próximo tipo apontado, isto é, se px é um ponteiro para uma variável int, depois de executar a instrução `px++;` o valor de px será incrementado de um int (quatro bytes em ambientes 32 bits).

Operações com Ponteiros

- Incrementando um ponteiro:
  - Cada vez que px é incrementado, apontará para o próximo int da memória.
  - A mesma ideia é verdadeira para decremento.
  - Toda vez que px é decrementado de uma unidade, o será de um tipo apontado, nesse caso int.
  - Você pode subtrair ou adicionar números de ponteiros. A instrução: `px = py + 5;`
  - Fará com que px caminhe cinco inteiros adiante de py.

Operações com Ponteiros

- Diferença:
  - Você pode encontrar a diferença entre dois ponteiros.
  - Essa diferença será expressa em número de variáveis apontadas entre eles.
  - Então, se py contém o endereço 4000 e px, 3996, a expressão `px - py;` resultará 1 quando px e py são ponteiros para int em ambientes de 32 bits.

Operações com Ponteiros

- Comparações entre Ponteiros:
  - Testes relacionais com >, <, >=, <=, ==, ou != são aceitos somente entre ponteiros do mesmo tipo.
  - Cuidado, se você comparar ponteiros que apontam para variáveis de tipos diferentes obterá resultados sem sentido.
  - Testes com NULL ou zero também podem ser feitos, contanto que você use o modificador de tipo:

```
if(px == (int *)0)
```

Roteiro

- O que são ponteiros?
- Por que os ponteiros são usados?
- Ponteiros variáveis
- Ponteiros constantes e o operador de endereço &
- Passando argumentos por referência com ponteiros
- Variáveis que armazenam endereços
- O operador indireto *
- Passando endereços para a função
- Permutação do valor de duas variáveis
- Ponteiros sem funções
- Ponteiros e variáveis apontadas
- Operações com ponteiros
- A unidade adotada em operações com ponteiros

A Unidade Adotada em Operações com Ponteiros

- Quando declaramos um ponteiro, o compilador necessita conhecer o tipo da variável apontada para poder executar corretamente operações aritméticas.

```
int *pi;
double *pd;
float *pf;
```

- O tipo declarado é entendido como o tipo de variável apontada.

A Unidade Adotada em Operações com Ponteiros

- Assim, se somarmos 1 a pi, estaremos somando o tamanho de um int; se somarmos 1 a pd, estaremos somando o tamanho de um double (oito bytes), e assim por diante.
- A unidade com ponteiros é o número de bytes do tipo apontado.

Referências

- MIZRAHI, Victorine V. **Treinamento em Linguagem C**, São Paulo: Pearson Prentice Hall, 2008;